Centro Universitário de Excelência
Sistemas de Informação

Refatoração e Padrão de Projetos

Autor: Ian Dias Duque
Olavo Barros Meira Filho

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é esclarecer a refatoração do projeto

1. Item 1:
Diagrama de Classes

2. Item 2:
Diagrama de Atividade

3. Item 3:
Builder

4. Item 4:
DTO

5. Item 5:
Controller

6. Item 6:
Mapper

DIAGRAMA DE CLASSES

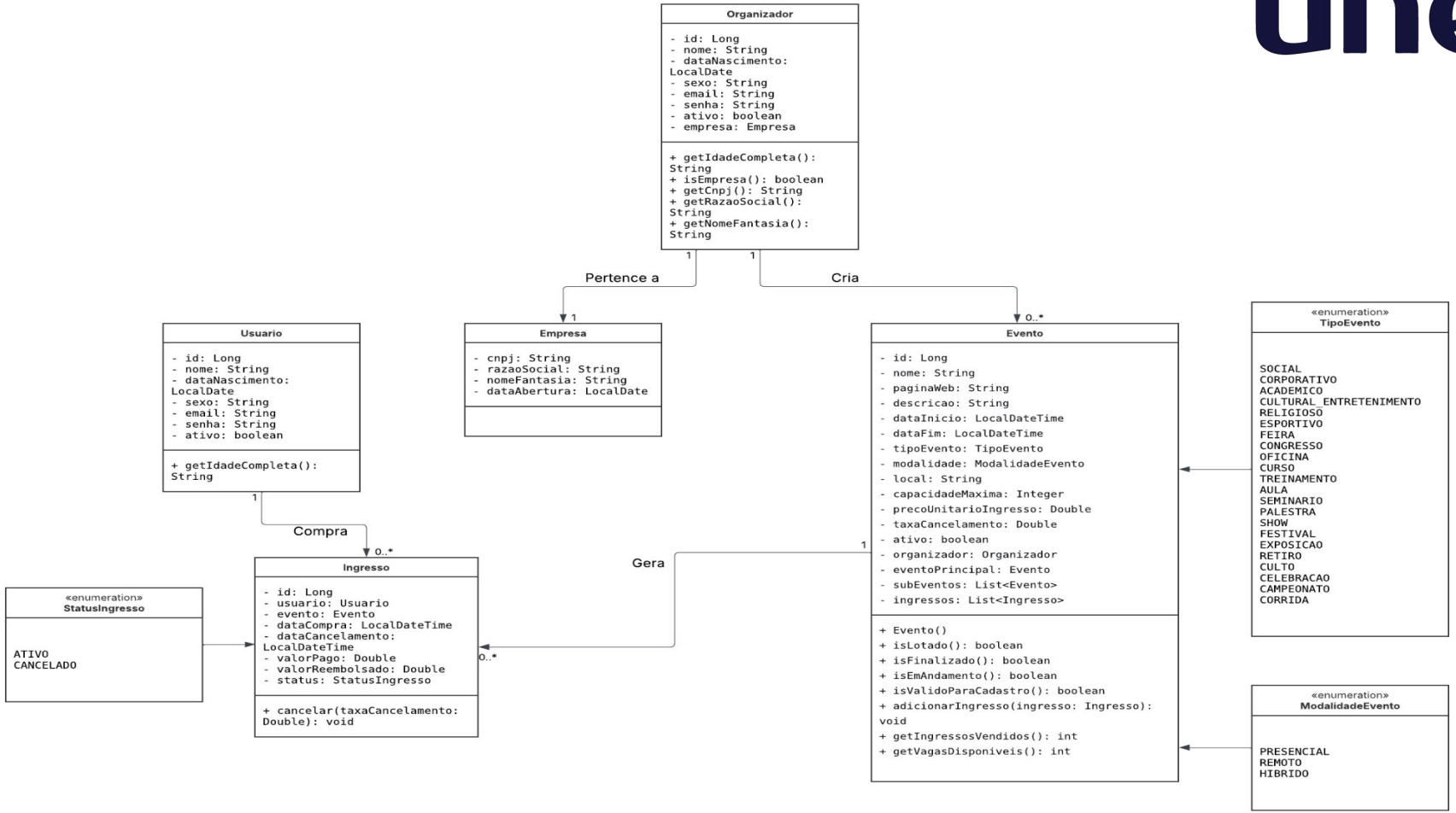
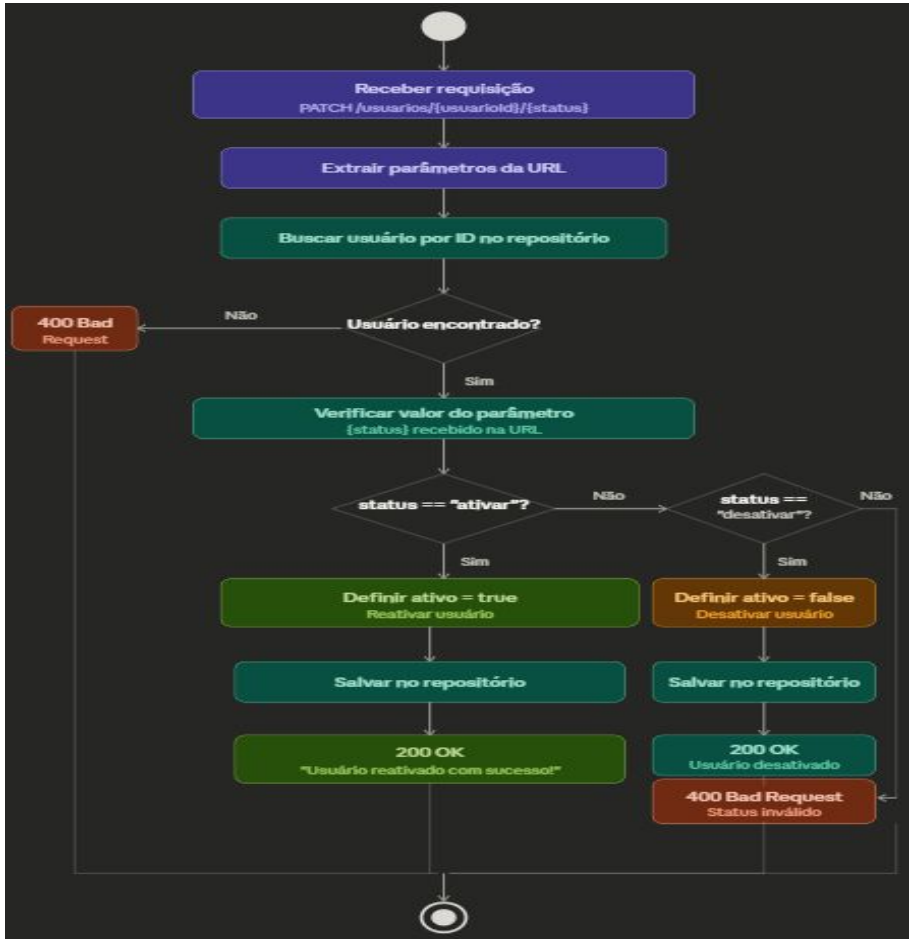


DIAGRAMA DE ATIVIDADE



PADRÃO BUILDER

- O Builder é um padrão de projeto criacional, que permite a construção de objetos complexos passo a passo.
- Ao utilizar o padrão Builder, você pode construir objetos de forma organizada e legível, evitando a confusão causada por muitos construtores e métodos setters.

PADRÃO BUILDER

```java
public class EventoBuilder {

    private Long id;
    private String nome;
    private String paginaWeb;
    private String descricao;
    private LocalDateTime dataInicio;
    private LocalDateTime dataFim;
    private TipoEvento tipoEvento;
    private ModalidadeEvento modalidade;
    private String local;
    private Integer capacidadeMaxima;
    private Double precoUnitarioIngresso;
    private Double taxaCancelamento;
    private boolean ativo;
    private Organizador organizador;
    private Evento eventoPrincipal;

    private EventoBuilder() {}

    public static EventoBuilder builder() { return new EventoBuilder(); }

    public EventoBuilder id(Long id) {
        this.id = id;
        return this;
    }
```

PADRÃO BUILDER

```java
    public Evento build() {
        Evento evento = new Evento();
        evento.setId(this.id);
        evento.setNome(this.nome);
        evento.setPaginaWeb(this.paginaWeb);
        evento.setDescricao(this.descricao);
        evento.setDataInicio(this.dataInicio);
        evento.setDataFim(this.dataFim);
        evento.setTipoEvento(this.tipoEvento);
        evento.setModalidade(this.modalidade);
        evento.setLocal(this.local);
        evento.setCapacidadeMaxima(this.capacidadeMaxima);
        evento.setPrecoUnitarioIngresso(this.precoUnitarioIngresso);
        evento.setTaxaCancelamento(this.taxaCancelamento);
        evento.setAtivo(this.ativo);
        evento.setOrganizador(this.organizador);
        evento.setEventoPrincipal(this.eventoPrincipal);
        return evento;
    }
}
```

PADRÃO DTO

- O padrão Data Transfer Object (DTO) é usado para transferir dados entre diferentes camadas de uma aplicação.

PADRÃO DTO

```java
public class EventoRequestDTO {  7 usages
    private String nome;  2 usages
    private String paginaWeb;  2 usages
    private String descricao;  2 usages
    private LocalDateTime dataInicio;  2 usages
    private LocalDateTime dataFim;  2 usages
    private TipoEvento tipoEvento;  2 usages
    private ModalidadeEvento modalidade;  2 usages
    private String local;  2 usages
    private Integer capacidadeMaxima;  2 usages
    private Double precoUnitarioIngresso;  2 usages
    private Double taxaCancelamento;  2 usages

    private Long eventoPrincipalId;  2 usages

    public EventoRequestDTO() {}  no usages

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getPaginaWeb() { return paginaWeb; }  3 usages
    public void setPaginaWeb(String paginaWeb) { this.paginaWeb = paginaWeb; }  no usages

    public String getDescricao() { return descricao; }  3 usages
    public void setDescricao(String descricao) { this.descricao = descricao; }  no usages

    public LocalDateTime getDataInicio() { return dataInicio; }  7 usages
    public void setDataInicio(LocalDateTime dataInicio) { this.dataInicio = dataInicio; }  no usages
```

PADRÃO DTO

```java
public class EventoResponseDTO {  8 usages
    private Long id;  2 usages
    private String nome;  2 usages
    private String paginaWeb;  2 usages
    private String descricao;  2 usages
    private String dataInicio;  2 usages
    private String dataFim;  2 usages
    private TipoEvento tipoEvento;  2 usages
    private ModalidadeEvento modalidade;  2 usages
    private String local;  2 usages
    private Integer capacidadeMaxima;  2 usages
    private Double precoUnitarioIngresso;  2 usages
    private Double taxaCancelamento;  2 usages
    private boolean ativo;  2 usages

    private int ingressosVendidos;  2 usages
    private int vagasDisponiveis;  2 usages

    private String organizador;  2 usages

    private Long eventoPrincipalId;  2 usages
    private String eventoPrincipalNome;  2 usages

    public EventoResponseDTO() {}  1 usage

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
```

CONTROLLER

- O Controller é responsável por receber as requisições HTTP da API e retornar a resposta ao cliente.

CONTROLLER

```java
@PatchMapping(path = "/{usuarioId}/{status}")  no usages
public ResponseEntity<String> alterarStatus(
        @PathVariable(parameter = "usuarioId") long usuarioId,
        @PathVariable(parameter = "status") String status) {

    Optional<Usuario> usuarioOpt = repositorio.buscarUsuarioPorId(usuarioId);

    if (!usuarioOpt.isPresent()) {
        return ResponseUtils.badRequest("Usuário não encontrado com ID: " + usuarioId);
    }

    Usuario usuario = usuarioOpt.get();

    if ("ativar".equalsIgnoreCase(status)) {
        usuario.setAtivo(true);
        repositorio.salvarUsuario(usuario);
        return ResponseUtils.ok("Usuário reativado com sucesso!");
    } else if ("desativar".equalsIgnoreCase(status)) {
        usuario.setAtivo(false);
        repositorio.salvarUsuario(usuario);
        return ResponseUtils.ok("Usuário desativado com sucesso!");
    } else {
        return ResponseUtils.badRequest("Status inválido. Use 'ativar' ou 'desativar'");
    }
}
```

MAPPER

- O Mapper é utilizado para converter objetos DTO em entidades e vice-versa.

MAPPER

```java
public class UsuarioMapper {  4 usages

    private UsuarioMapper() {  no usages

    }


    public static Usuario toModel(UsuarioRequestDTO dto) {  1 usage
        if (dto == null) return null;

        Usuario usuario = new Usuario();
        usuario.setNome(dto.getNome());
        usuario.setDataNascimento(dto.getDataNascimento());
        usuario.setSexo(dto.getSexo());
        usuario.setEmail(dto.getEmail());
        usuario.setSenha(dto.getSenha());
        return usuario;
    }


    public static void updateModel(UsuarioRequestDTO dto, Usuario usuarioExistente) {  1 usage
        if (dto == null || usuarioExistente == null) return;

        usuarioExistente.setNome(dto.getNome());
        usuarioExistente.setDataNascimento(dto.getDataNascimento());
        usuarioExistente.setSexo(dto.getSexo());
```

MAPPER

```java
public class UsuarioMapper {  4 usages
    public static void updateModel(UsuarioRequestDTO dto, Usuario usuarioExistente) {  1 usage

        usuarioExistente.setNome(dto.getNome());
        usuarioExistente.setDataNascimento(dto.getDataNascimento());
        usuarioExistente.setSexo(dto.getSexo());

        if (dto.getSenha() != null && !dto.getSenha().isEmpty()) {
            usuarioExistente.setSenha(dto.getSenha());
        }
    }


    public static UsuarioResponseDTO toResponse(Usuario usuario) {  1 usage
        if (usuario == null) return null;

        UsuarioResponseDTO dto = new UsuarioResponseDTO();
        dto.setId(usuario.getId());
        dto.setNome(usuario.getNome());
        dto.setDataNascimento(usuario.getDataNascimento() != null
                ? usuario.getDataNascimento().toString() : null);
        dto.setIdadeCompleta(usuario.getIdadeCompleta());
        dto.setSexo(usuario.getSexo());
        dto.setEmail(usuario.getEmail());
        dto.setAtivo(usuario.isAtivo());
        return dto;
    }
}
```